МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ Федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»
Московский приборостроительный техникум

ОТЧЕТ

по учебной практике

УП 01.02 «Учебная практика»
индекс по УП и наименование практики

Профессионального модуля ПМ.01 «Разработка модулей программного обеспечения для компьютерных систем»
индекс по УП и наименование профессионального модуля

Специальность 09.02.07 «Информационные системы и программирование»
код и наименование специальности

Студент **Папазов Максим Юрьевич**
(фамилия, имя, отчество)

Группа **П50-3-24**

Руководитель по практической подготовке от техникума **Юшина Дарья Денисовна**
(фамилия, имя, отчество)

«___» _____ 2026 года

Москва — 2026

СОДЕРЖАНИЕ

Лабораторная работа №1 «Компонент ProductCard: props и локальное состояние»

Практической работе № 2

Веб-приложение магазина услуг с серверным API, авторизацией, корзиной и системой скидок

Лабораторная работа №1

«Создание компонента ProductCard с использованием props и хука useState»

Цель работы

Научиться создавать переиспользуемые React-компоненты, которые получают данные через **props** и управляют внутренним состоянием с помощью хука **useState**. На примере карточки товара для магазина игровых девайсов освоить передачу названия, цены и изображения из родительского компонента, реализовать кнопку «Добавить в корзину» со счётчиком и отрендерить несколько экземпляров компонента на странице.

Задачи

- Установить окружение: Visual Studio Code и Node.js (LTS) — проверить **node -v** и **npm -v**.
- Создать проект на **Vite + React** и настроить базовую структуру **src/**.
- Реализовать компонент **ProductCard**, принимающий **name**, **price**, **image** через props.
- Добавить локальное состояние **count** через **useState(0)** и кнопку «Добавить в корзину» (+1 к счётчику).
- Отрендерить 5–6 карточек в **App.jsx** с разными товарами (массив-заглушка).
- Оформить карточки (CSS): сетка, тени, адаптив; проверить сборку **npm run build**.

Краткие теоретические сведения

Props — входные параметры компонента, аналогичные аргументам функции. Родитель передаёт **<ProductCard name="Геймпад" price={7990} image="/images/gamepad.svg" />**, а внутри компонент читает их как **props.name**. Props неизменяемы (immutable) — компонент не должен их менять.

useState — хук, который добавляет компоненту память: **const [count, setCount] = useState(0)**. При вызове **setCount(count + 1)** React перерисовывает компонент с новым значением. Состояние изолировано: у каждой карточки свой независимый счётчик.

На данном этапе данные хранятся в локальном массиве **products** — СУБД будет выбрана позже при подключении API, как указано в задании.

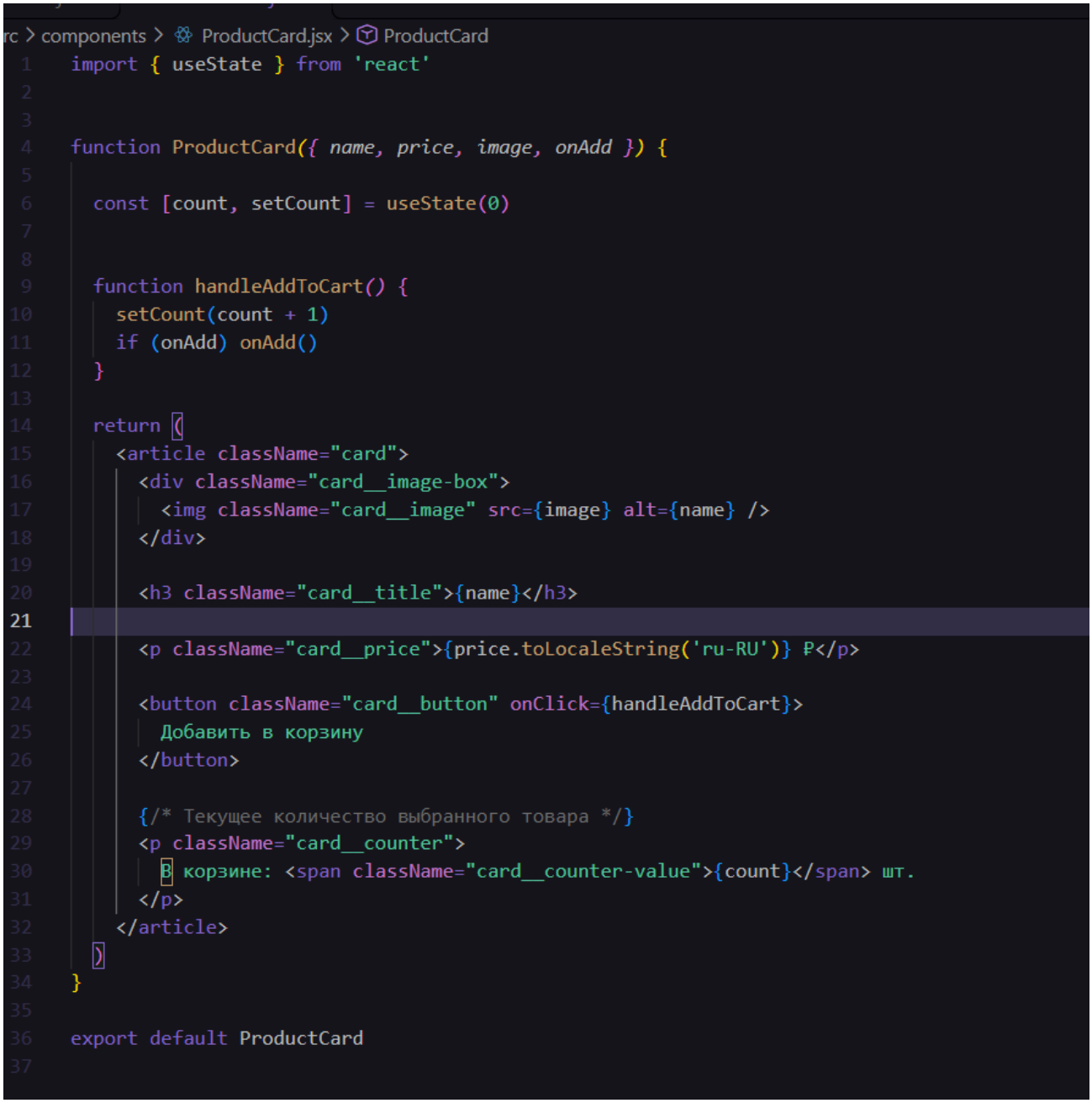
Ход работы

1. Создание проекта

Проект создан командой **npm create vite@latest gaming-shop ---template react**. Установлены зависимости **react** и **react-dom**. Структура: **index.html** → **src/main.jsx** → **App.jsx**.

2. Компонент ProductCard (первая версия — с useState)

Компонент получает три пропса и хранит количество добавлений. Кнопка увеличивает счётчик, а при **count > 0** под ней появляется строка «В корзине: N шт.».



Счётчик реализован именно через **useState**, как требовало задание лабораторной №1. На этом этапе каждая карточка считает независимо — общее состояние появится в работе №2.

3. Рендер нескольких карточек в App.jsx

rc > components > ProductCard.jsx > ProductCard

```jsx
import { useState } from 'react'


function ProductCard({ name, price, image, onAdd }) {

  const [count, setCount] = useState(0)


  function handleAddToCart() {
    setCount(count + 1)
    if (onAdd) onAdd()
  }

  return (
    <article className="card">
      <div className="card__image-box">
        <img className="card__image" src={image} alt={name} />
      </div>

      <h3 className="card__title">{name}</h3>

      <p className="card__price">{price.toLocaleString('ru-RU')} ₽</p>

      <button className="card__button" onClick={handleAddToCart}>
        Добавить в корзину
      </button>

      {/* Текущее количество выбранного товара */}
      <p className="card__counter">
        В корзине: <span className="card__counter-value">{count}</span> шт.
      </p>
    </article>
  )
}

export default ProductCard
```

Рисунок 1 — Код компонента ProductCard (вторая версия: props + useCart + navigate) и фрагмент App.jsx

Результат

Собрано приложение с 6 карточками игровых девайсов. Каждая карточка корректно отображает имя, цену и изображение, кнопка «Добавить в корзину» увеличивает локальный счётчик. Сборка **npm run build** проходит без ошибок, проект открывается на **http://localhost:5173/**.

Вывод: изучены принципы **props** и **useState** в React. Освоена передача данных от родителя к дочернему компоненту, работа с локальным состоянием и рендер списков через **map()** с ключом **key**. Получен базовый UI-кит, который в следующей работе будет расширен до SPA с глобальным состоянием.

Цель и задачи работы

Цель работы — разработать одностраничное приложение (SPA) на React для магазина игровых девайсов. Приложение должно отображать каталог товаров, предоставлять общую корзину, переключать светлую и тёмную темы и обеспечивать переходы между страницами без полной перезагрузки браузера.

В ходе работы требовалось реализовать каталог не менее чем из пяти товаров; добавление товара в глобальное состояние; страницу корзины с удалением позиций и подсчётом суммы; тему через Context API; меню на NavLink с подсветкой активной страницы; программные переходы после добавления товара и успешного входа; управление корзиной через Context API и useReducer.

Использованные технологии

Для реализации использованы React 18, JavaScript (ES6+), Vite, React Router 6, Context API, хук useReducer, CSS-переменные и localStorage. Данные товаров хранятся локально в src/data/products.js.

Подключение СУБД и внешнего API в рамках задания не выполнялось.

React отвечает за компоненты и состояние интерфейса, React Router — за маршруты и переходы SPA, Context API предоставляет глобальные данные компонентам, а useReducer централизует операции над корзиной.

Каталог и добавление товаров

Массив products содержит девять игровых товаров. Каждая запись включает идентификатор, название, цену, путь к изображению, категорию и дополнительные поля. Страница каталога выводит карточки, позволяет искать товары, фильтровать их по категориям и сортировать список.

ProductCard получает объект товара через props. При нажатии «В корзину» компонент вызывает addItem(product) из useCart(). При включённом параметре redirectAfterAdd выполняется программный переход на /cart с помощью useNavigate().

```
   function handleAdd() {   addItem(product)
showToast(`«${product.name}» в корзине`)   if
(redirectAfterAdd) navigate('/cart') }
```



```
}

return (
  <article className="card">
    <div className="card__image-box">
      {product.tag && <span className="card__tag">{product.tag}</span>}
      <img className="card__image" src={product.image} alt={product.name} />
    </div>

    <div className="card__body">
      <span className="card__category">{product.category}</span>


      <Link to={`/product/${product.id}`} className="card__title-link">
        <h3 className="card__title">{product.name}</h3>
      </Link>

      <p className="card__rating">★ {product.rating.toFixed(1)}</p>

      <p className="card__price">
        {product.oldPrice && (
          <span className="card__old-price">{product.oldPrice.toLocaleString('ru-RU')} ₽</span>
        )}
        {product.price.toLocaleString('ru-RU')} ₽
      </p>

      <div className="card__actions">
        <button className="btn btn--block" onClick={handleAdd}>
```

Иллюстрация 1 — Каталог товаров и карточки игровых устройств

Глобальная корзина и вычисление суммы

Корзина реализована на Context API и useReducer. Начальное состояние содержит массив items. Reducer обрабатывает действия ADD_ITEM, INCREASE_ITEM, DECREASE_ITEM, REMOVE_ITEM и CLEAR_CART. Повторное добавление существующего товара увеличивает его количество, а новый товар добавляется с qty = 1.

Итоговые значения вычисляются на основании глобального массива: totalCount — общее число единиц, totalSum — сумма цены каждой позиции с учётом её количества. CartPage показывает название, цену, количество и кнопку «Удалить» для каждой строки. Сумма пересчитывается после каждого изменения и выводится как в корзине, так и в ссылке «Корзина» верхнего меню. Состояние корзины дополнительно сохраняется в localStorage.

Переключение и сохранение темы

ThemeContext предоставляет текущую тему и функцию toggleTheme(). Значение темы задаётся атрибутом data-theme элемента html. CSS-переменные определяют фон, цвет текста, поверхности карточек, акцентные элементы и границы.

При первом открытии ThemeProvider считывает сохранённое значение. После переключения useEffect применяет тему и записывает её в localStorage, поэтому выбранный режим восстанавливается после обновления страницы.

Вывод

В ходе практической работы разработано SPA-приложение магазина игровых девайсов на React. Реализованы каталог из девяти товаров, глобальная корзина на Context API и useReducer, отображение позиций и итоговой суммы, светлая и тёмная темы с сохранением выбора, навигация через React Router, защита страницы кабинета и программные переходы. Сборка и автоматические проверки завершились успешно. Полученные навыки можно применить при дальнейшем подключении API и СУБД.

Лабораторная работа № 2

Цель и задачи работы

Цель работы — разработать одностраничное приложение (SPA) на React для магазина игровых девайсов. Приложение должно отображать каталог товаров, предоставлять общую корзину, переключать светлую и тёмную темы и обеспечивать переходы между страницами без полной перезагрузки браузера.

В ходе работы требовалось реализовать каталог не менее чем из пяти товаров; добавление товара в глобальное состояние; страницу корзины с удалением позиций и подсчётом суммы; тему через Context API; меню на NavLink с подсветкой активной страницы; программные переходы после добавления товара и успешного входа; управление корзиной через Context API и useReducer.

Использованные технологии

Для реализации использованы React 18, JavaScript (ES6+), Vite, React Router 6, Context API, хук useReducer, CSS-переменные и localStorage. Данные товаров хранятся локально в src/data/products.js.

Подключение СУБД и внешнего API в рамках задания не выполнялось.

React отвечает за компоненты и состояние интерфейса, React Router — за маршруты и переходы SPA, Context API предоставляет глобальные данные компонентам, а useReducer централизует операции над корзиной.

Каталог и добавление товаров

Массив products содержит девять игровых товаров. Каждая запись включает идентификатор, название, цену, путь к изображению, категорию и дополнительные поля. Страница каталога выводит карточки, позволяет искать товары, фильтровать их по категориям и сортировать список.

ProductCard получает объект товара через props. При нажатии «В корзину» компонент вызывает addItem(product) из useCart(). При включённом параметре redirectAfterAdd выполняется программный переход на /cart с помощью useNavigate().



```
const inCart = items.find((item) => item.id === product.id)
const qty = inCart ? inCart.qty : 0

function handleAdd() {
  addItem(product)
  showToast(`«${product.name}» в корзине`)

  if (redirectAfterAdd) navigate('/cart')
}


function handleDetails() {
  navigate(`/product/${product.id}`)
}
```

Иллюстрация 1 — Каталог товаров и карточки игровых устройств

Глобальная корзина и вычисление суммы

Корзина реализована на Context API и useReducer. Начальное состояние содержит массив items. Reducer обрабатывает действия ADD_ITEM, INCREASE_ITEM, DECREASE_ITEM, REMOVE_ITEM и CLEAR_CART. Повторное добавление существующего товара увеличивает его количество, а новый товар добавляется с qty = 1.

Итоговые значения вычисляются на основании глобального массива: totalCount — общее число единиц, totalSum — сумма цены каждой позиции с учётом её количества. CartPage показывает название, цену, количество и кнопку «Удалить» для каждой строки. Сумма пересчитывается после каждого изменения и выводится как в корзине, так и в ссылке «Корзина» верхнего меню. Состояние корзины дополнительно сохраняется в localStorage.

```
function cartReducer(state, action) {
    case 'ADD_ITEM': {
      const product = action.payload
      const exists = state.items.find((item) => item.id === product.id)

      if (exists) {
        return {
          ...state,
          items: state.items.map((item) =>
            item.id === product.id ? { ...item, qty: item.qty + 1 } : item,
          ),
        }
      }

      return {
        ...state,
        items: [
          ...state.items,
          {
            id: product.id,
            name: product.name,
            price: product.price,
            image: product.image,
            qty: 1,
          },
        ],
      }
    }
```

Иллюстрация 2 — Страница корзины, список позиций и итоговая сумма

Переключение и сохранение темы

ThemeContext предоставляет текущую тему и функцию toggleTheme(). Значение темы задаётся атрибутом data-theme элемента html. CSS-переменные определяют фон, цвет текста, поверхности карточек, акцентные элементы и границы.

При первом открытии ThemeProvider считывает сохранённое значение. После переключения useEffect применяет тему и записывает её в localStorage, поэтому выбранный режим восстанавливается после обновления страницы.

```
export function ThemeProvider({ children }) {

  const [theme, setTheme] = useState(() => localStorage.getItem('gamedeck-theme') || 'dark')

  useEffect(() => {

    document.documentElement.dataset.theme = theme
    localStorage.setItem('gamedeck-theme', theme)
  }, [theme])

  const toggleTheme = () => setTheme((current) => (current === 'dark' ? 'light' : 'dark'))

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme, isDark: theme === 'dark' }}>
      {children}
    </ThemeContext.Provider>
  )
}
```

Иллюстрация 3 — Кнопка переключения светлой и тёмной темы

Навигация и программные переходы

Навигация реализована с помощью BrowserRouter, Routes, Route и NavLink. Основные маршруты: /, /catalog, /product/:id, /cart, /login и /dashboard. Компонент NavLink автоматически добавляет класс активной ссылке; CSS выделяет текущий пункт меню.

Для программной навигации используется useNavigate(). После добавления товара пользователь может быть направлен на /cart. Маршрут /dashboard защищён компонентом ProtectedRoute. Если пользователь не авторизован, компонент Navigate перенаправляет его на /login и сохраняет исходный адрес. После успешного демонстрационного входа LoginPage возвращает пользователя на запрошенную страницу, обычно /dashboard.

Вывод

В ходе практической работы разработано SPA-приложение магазина игровых девайсов на React. Реализованы каталог из девяти товаров, глобальная корзина на Context API и useReducer, отображение позиций и итоговой суммы, светлая и тёмная темы с сохранением выбора, навигация через React Router, защита страницы кабинета и программные переходы. Сборка и автоматические проверки завершились успешно. Полученные навыки можно применить при дальнейшем подключении API и СУБД.

Лабораторная работа № 3

Веб-приложение магазина услуг с серверным API, авторизацией, корзиной и системой скидок

1. Цель и задачи работы

Цель лабораторной работы — разработать веб-приложение для каталога услуг с корзиной и авторизацией, связанное с серверным REST API и базой данных Microsoft SQL Server.

В соответствии с заданием реализованы каталог услуг с описанием, длительностью и ценой; поиск, фильтрация по категориям и диапазону цен, сортировка; корзина и оформление записей; регистрация, вход, личный кабинет и история записей; общие скидки и персональные купоны. Администратор может добавлять категории и услуги, изменять параметры услуг, скидки и доступность, просматривать клиентов и все записи.

2. Технологии и структура проекта

Клиентская часть создана на React 18 и Vite и интегрирована в существующий магазин GameDeck как отдельный раздел GameCare; прежние страницы магазина сохранены. Для маршрутов используется React Router. Корзина управляется через Context API и useReducer. Сервер написан на Python с использованием http.server и адаптера pyodbc. Подключение к Microsoft SQL Server выполняется через ODBC Driver 18. Vite проксирует адреса /api на локальный сервер http://127.0.0.1:8000. Браузер обращается к относительному адресу /api, а запрос пересылается серверу разработки.

3. Проектирование базы данных

Сценарий sql/001_create_schema.sql создаёт базу GameCareDB и структуру в MS SQL Server. При запуске server.py добавляет начальные роли, категории, услуги и демо-пользователей. В дополнение к пяти основным таблицам задания предусмотрены Categories и Coupons для категорий и персональных скидок. Внешние ключи связывают таблицы и не позволяют создать запись, ссылающуюся на отсутствующего пользователя, роль, категорию или услугу. Для цены, скидки и количества заданы ограничения допустимых значений.

5. Каталог услуг и фильтрация категорий

Каталог загружает список услуг и категории параллельными запросами через Promise.all. Пока сервер отвечает, интерфейс показывает сообщение «Загружаем каталог услуг с сервера…». Если соединение отсутствует или сервер вернул ошибку, показывается текст ошибки и кнопка повторной загрузки.

Категории выводятся в боковой панели слева от карточек; пользователь может выбрать несколько категорий. Поле поиска ищет текст в названии и описании. Добавлены поля минимальной и максимальной цены (по цене с учётом скидки), сортировка по стоимости или названию, отображение количества найденных услуг и сброс параметров фильтрации. На узких экранах панель фильтров перестраивается над каталогом.

Карточка услуги показывает название, описание, длительность и цену. При наличии общей скидки отображаются старая цена, новая цена и процент скидки. Кнопка «Добавить в корзину» передаёт услугу в глобальное состояние.

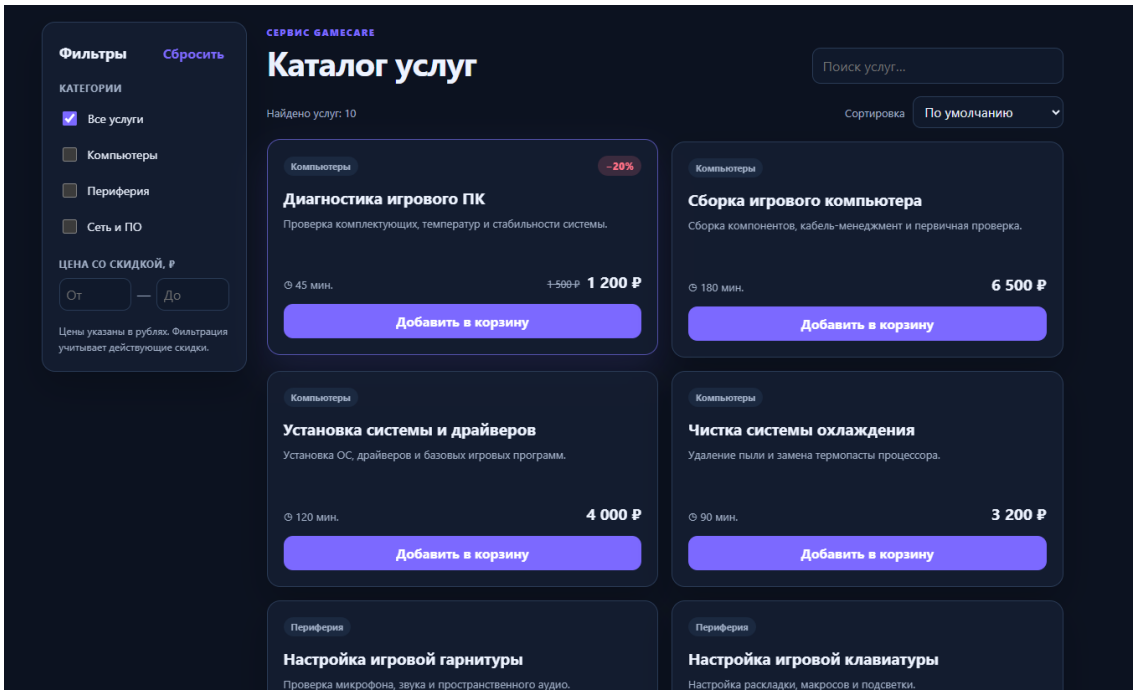


Рисунок 1 — Каталог услуг с боковой панелью категорий и флажками

6. Корзина, записи и платежи

Корзина хранится в глобальном Context с useReducer. Состояние содержит массив объектов: данные услуги и выбранное количество. Доступны действия ADD, REMOVE, CHANGE и CLEAR. Массив сохраняется в localStorage, поэтому содержимое остаётся при обновлении страницы

На странице корзины пользователь видит выбранные услуги, длительность, цену за единицу, количество, сумму строки и итог. Количество можно изменить кнопками «−» и «+», позицию — удалить. Персональный купон применяется к сумме со скидками услуг.

При оформлении клиент передаёт API только идентификаторы услуг и количество. Сервер повторно читает цены и скидки из базы и самостоятельно рассчитывает итог, не полагаясь на сумму, присланную браузером. Для каждой услуги создаётся запись Appointments и связанная строка Payments со статусом simulated. Это демонстрационная фиксация оплаты: реальные деньги не списываются.

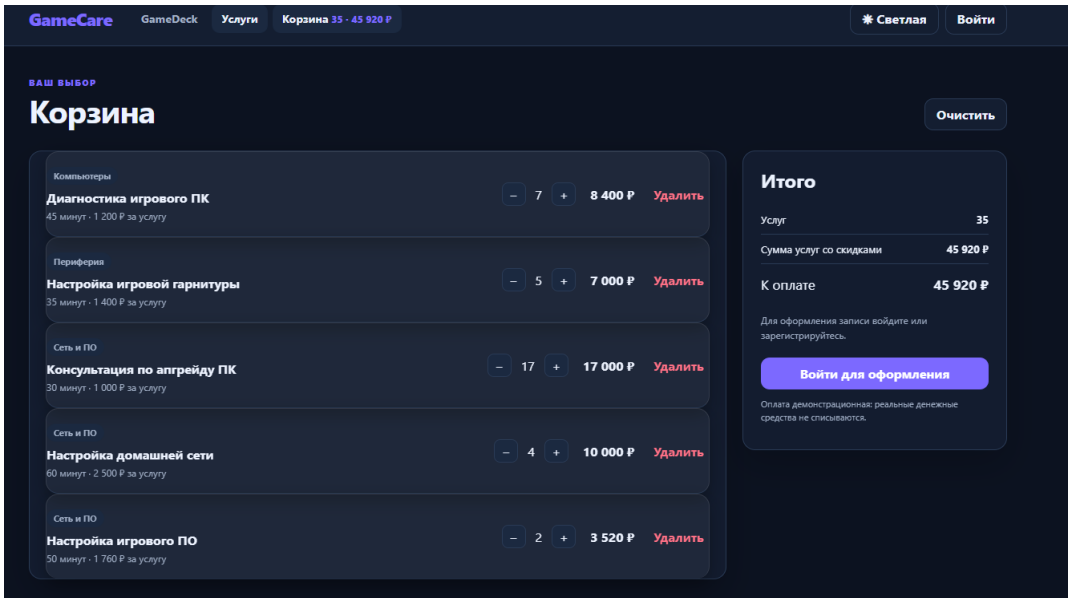


Рисунок 2 — Корзина с выбранными услугами, купоном и итоговой суммой

7. Скидки и панель администратора

В начальной базе заведено 10 услуг; у трёх из них стоит общая скидка 20%. Таким образом, скидкой охвачено ровно 30% каталога. Размер скидки администратор может менять в диапазоне от 0 до 90 процентов.

Демо-пользователю user@student.ru назначен купон WELCOME10 на 10%. Поле discount_percent хранится в таблице Coupons вместе с assigned_user_id, поэтому купон персональный. Администратор может изменить код и процент или указать 0, чтобы снять скидку.

Административная панель позволяет создавать категории и услуги, редактировать название, описание, длительность, цену, категорию, скидку и статус публикации. Также администратор управляет персональными купонами пользователей и просматривает все записи с контактами клиента, составом, датой, суммой и состоянием демонстрационной оплаты. Управление услугами и категориями ограничено ролью admin; новые и изменённые данные сохраняются в MS SQL Server.

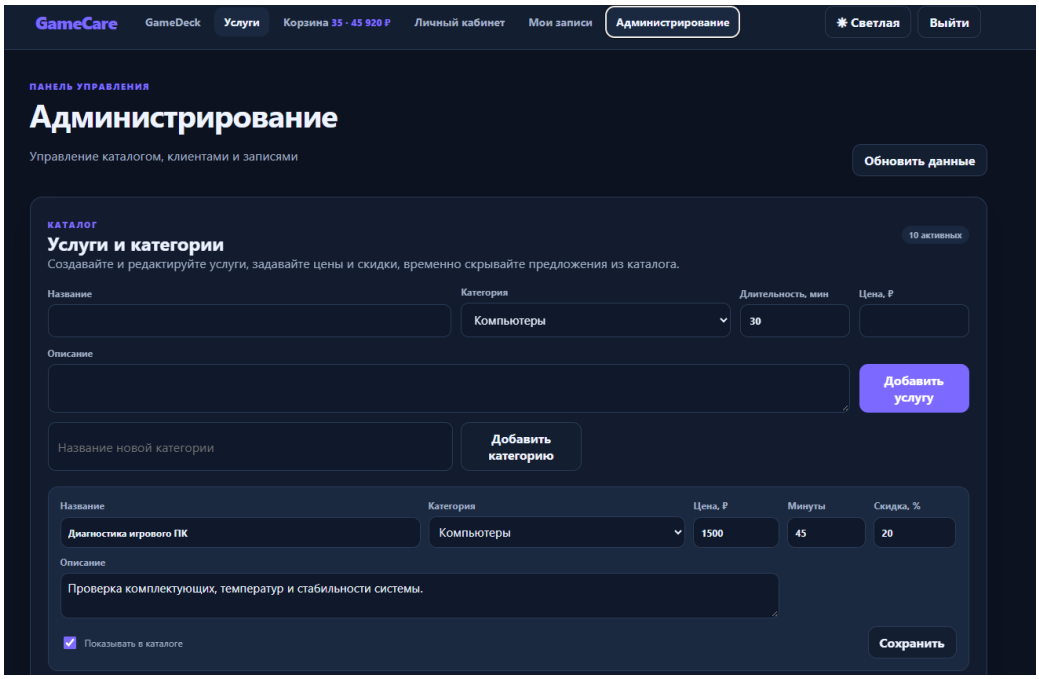


Рисунок 3 — Панель администратора: категории, общие скидки и купоны

8. Обработка ошибок и состояния загрузки

Клиентская функция apiRequest выполняет fetch к серверу, добавляет заголовок JSON и токен авторизации. Сетевой сбой преобразуется в понятное сообщение. Тело ответа разбирается как JSON; при неуспешном HTTP-коде выбрасывается ошибка с текстом сервера.

Каталог и панель администратора показывают индикатор загрузки. В формах авторизации проверяются заполненность полей и корректный формат email; сервер повторяет проверки и возвращает ошибки, например при неверном пароле, занятом email, дублирующейся категории или недостаточной длине пароля.

9. Проверка приложения

Проверены production-сборка клиентской части и синтаксис Python-сервера. В коде предусмотрены API-сценарии проверки каталога, фильтрации и сортировки, профиля, авторизации, оформления записей, скидок, управления услугами и просмотра заказов

администратором. Полный интеграционный прогон API выполняется после подключения локального Microsoft SQL Server и установки ODBC Driver 18.

В seed-данных предусмотрено 10 услуг, из них 3 имеют скидку; созданы начальные категории и демо-пользователи. После подключения MS SQL Server следует проверить вход и регистрацию, фильтрацию каталога, создание и редактирование услуг, создание записей и платёжных записей, персональные купоны и просмотр административного списка записей.

Production-сборка клиентской части повторно выполнена командой npm run build и завершилась успешно (65 модулей). Проверен Python-синтаксис сервера. Для сквозной проверки API требуется установленный и настроенный SQL Server с ODBC Driver 18; экземпляр SQL Server в среде подготовки отчёта не предоставлен.

Вывод

В ходе лабораторной работы создан отдельный раздел GameCare внутри существующего проекта GameDeck без замены исходных страниц магазина. Реализованы каталог услуг с категориями, поиском, фильтром цены и сортировкой; корзина, регистрация и вход, личный кабинет, история записей, роли пользователей, общие скидки и персональные купоны. Администратор управляет категориями, услугами, клиентскими купонами и просматривает записи. Серверное API и база данных используют Microsoft SQL Server. Production-сборка клиентской части проверена; сквозной запуск требует локального экземпляра MS SQL Server и ODBC Driver 18.